

**《面向对象程序设计》
实验报告
——学生成长档案应用程序**

班级：数据 231

学号：235127

姓名：艾洋

完成时间：2024 年 11月 22日

目录

实验一：功能分析与设计	3
实验二：类与对象设计	4
实验三：继承性与多态性设计	5
实验四：模板设计	5
实验五：异常处理设计	6
附录一：核心代码	8
附录二：实验结果截屏及实验成果分析	15

河北工业大学课程共享计划

实验一：功能分析与设计

1.1 功能分析

该程序的目标是设计一个**学生成长档案管理系统**，用于记录和查询学生在各方面的成长情况，包括学业成绩、能力发展、休闲娱乐等。系统的主要功能包括：

学生信息录入：输入学生基本信息、成绩、能力等。

查询学生信息：展示所有学生的档案。

查看科目平均分：能够计算并显示每门课程的平均成绩。

1.2 功能设计

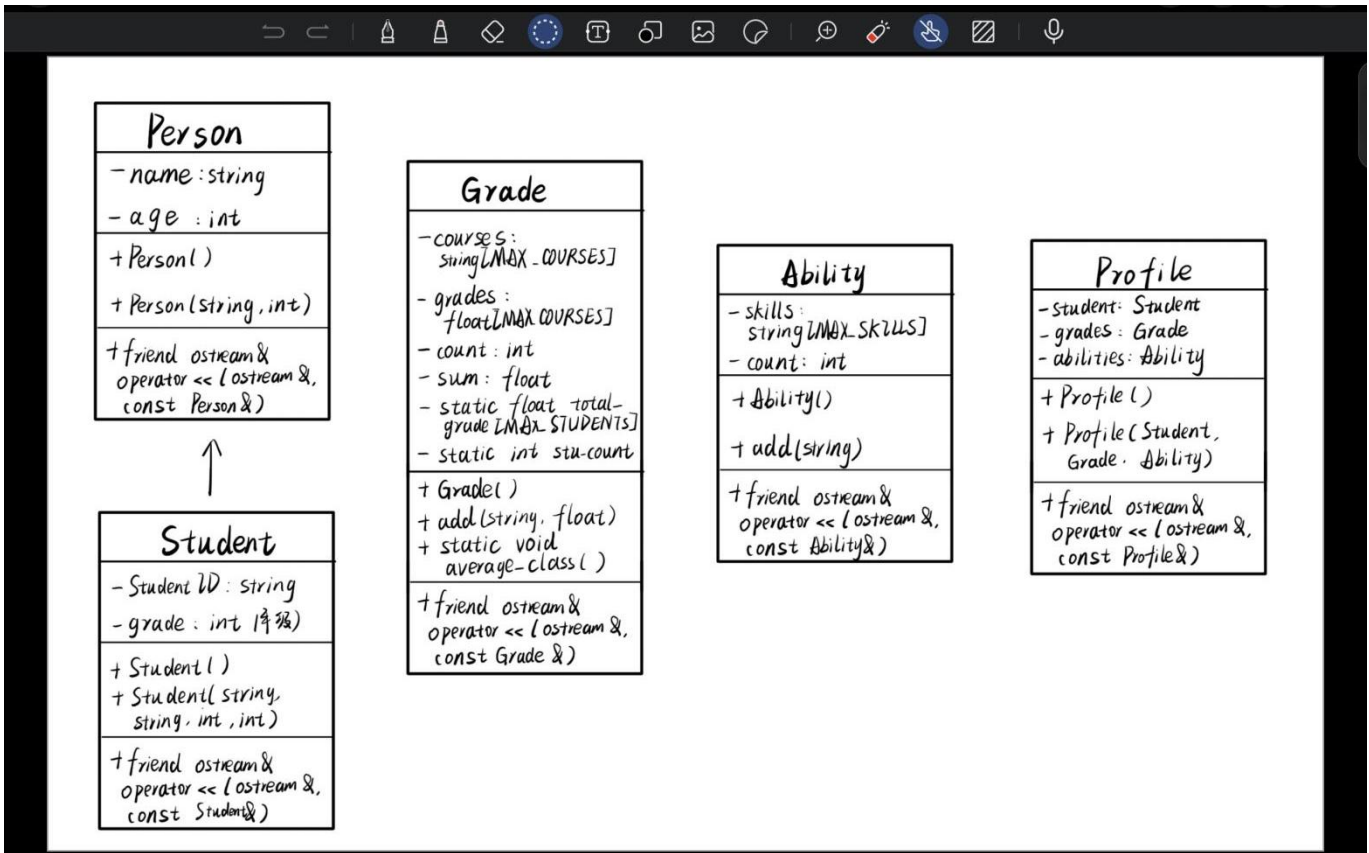
学生档案：每个学生的档案包含基本信息（姓名、学号、年龄、年级），成绩信息（课程名、成绩），以及能力信息（例如：英语口语、编程能力）。

成绩计算：支持对学生各科成绩的输入，能够计算各个科目的平均分。

学生能力展示：记录学生的技能或兴趣爱好等，展示他们的能力发展情况。

输入验证：对于学号、成绩等输入进行有效性检查，确保数据的准确性。

实验二：类与对象设计



2.1 类设计

为了实现该系统的功能，我们设计了几个关键类：

Person 类：存储学生的基本信息，如姓名和年龄。

Student 类：继承自 Person 类，添加了学号和年级信息，构成学生档案。

Grade 类：存储学生的成绩信息，并提供计算每门课程平均分的功能。

Ability 类：记录学生的能力信息，如编程能力、语言能力等。

Profile 类：封装学生的档案，包括学生基本信息、成绩单和能力卡片。

通过这些类的设计，学生的基本信息、成绩和能力都可以进行独立的管理，并且能够提供简洁的接口进行信息查询。

2.2 类的实现与功能

Person 类：基本构造函数和成员函数用于存储和输出个人信息。

Student 类：在继承 Person 类的基础上，扩展了学号和年级信息，并提供输出学生详细信息的功能。

Grade 类：通过数组存储课程成绩，并实现计算每门课程平均分的静态方法。

Ability 类：存储学生的能力信息，能够方便地记录和输出学生的各项技能。

Profile 类：将学生信息、成绩单和能力卡片组合在一起，实现学生成长档案的整体展示。

实验三：继承性与多态性设计

3.1 继承性设计

在实验二中，我们实现了 `Student` 类继承自 `Person` 类，能够继承基本信息并扩展特有的属性和行为。继承的优点是能够重用已有类的功能，同时增加新的属性和方法。（类图同实验二）

3.2 多态性设计

在学生信息的展示过程中，运用了多态性设计。通过重载运算符<<实现不同类对象的输出操作。这样可以让学生信息、成绩信息、能力信息等在不同的上下文中方便地进行输出。

实验四：模板设计（使用了 STL）

为了提高代码的灵活性和重用性，我们在系统中使用了模板技术，尤其是在处理学生成绩分析时。通过函数模板或类模板，我们能够根据不同的数据类型（如整型或浮点型）来进行成绩统计，避免了多次重复编写类似的代码。与传统的数组方式相比，使用 `vector` 容器可以更加方便地动态管理学生成绩数据，且无需考虑固定大小的问题。

例如，针对不同类型成绩的统计，使用函数模板可以使代码更加通用和简洁，同时使用 `vector` 代替数组，更能灵活应对动态数据。我们可以通过 `vector<T>` 来处理不同数据类型的成绩，避免了硬编码数组大小的问题。

实验五：异常处理设计

5.1 异常处理的应用

为了确保输入数据的合法性，我们在程序中加入了异常处理机制。特别是学号输入的部分，如果学号大于 25000，则会抛出自定义异常 `InvalidIDException`，并提示用户输入错误。这种机制提高了程序的鲁棒性，避免了无效数据的录入。

5.2 异常示例：

```
cpp
复制代码
try {
    if (stoi(id) > 25000) {
        throw InvalidIDException("学号不能大于 25000!");
    }
} catch (const InvalidIDException& e) {
    cout << "输入错误：" << e.what() << endl;
} catch (const exception& e) {
    cout << "发生异常：" << e.what() << endl;
}
```

在此代码段中，若学号超过了规定的最大值，系统将抛出异常并显示错误信息。

5.2 异常处理流程图



结论

通过这次实验，实现了一个完整的 C++ 风格学生成长档案应用程序，并通过面向对象的设计方法，将学生的基本信息、成绩、能力等数据进行封装和管理。系统中充分应用了类的封装、继承、多态、模板和异常处理等面向对象的特性，能够灵活地应对不同类型的数据输入输出和异常情况。

通过设计和实现这个系统，不仅深入理解了 C++ 的面向对象特性，还提高了在实际项目中应用这些特性的能力。

附录一：核心代码

头文件Student development file.h

```
#include <iostream>

#include <string>

#include <stdexcept>

using namespace std;

const int MAX_COURSES = 5;
const int MAX_SKILLS = 5;
const int MAX_STUDENT = 5;

class Person {
protected:
    string name;
    int age;

public:
    Person() : name("无名"), age(0) {}
    Person(string nm, int a) : name(nm), age(a) {}

    friend ostream& operator<<(ostream& os, const Person& p) {
        os << "name: " << p.name << endl
        << "age: " << p.age << endl;
        return os;
    }
};

class Student : public Person {
private:
    string studentID;
    int grade;

public:
```

```

Student() : studentID("未知"), grade(0) {}

Student(string id, string nm, int a, int g) : Person(nm, a), studentID(id), grade(g)
{}

friend ostream& operator<<(ostream& os, const Student& s) {
    os << "--基本信息--" << endl
    << "studentID: " << s.studentID << endl;
    os << static_cast<const Person&>(s); // 调用 Person 的输出流运算符
    os << "grade: " << s.grade << endl << endl;
    return os;
}

};

class Grade {
private:
    string courses[MAX_COURSES];
    float grades[MAX_COURSES];
    int count = 0;
    float sum = 0;
    static float total_grade[MAX_STUDENT];
    static int stu_count;

public:
    Grade() : count(0) {}

    void add(string course, float grade) {
        if (count < MAX_COURSES) {
            courses[count] = course;
            grades[count] = grade;
            sum += grade;
            total_grade[count] += grades[count];
            count++;
            stu_count++;
        }
    }
}

```

```

static void average_class() {
    cout << endl << "Class Grade:" << endl << "--每门课程的平均分--" << endl;
    for (int i = 0; i < 3; i++) {
        if (stu_count > 0) {
            cout << "课程" << (i + 1) << "的平均分: " << (total_grade[i] / (stu_count / 3))
<< endl;
        }
    }
}

friend ostream& operator<<(ostream& os, const Grade& g) {
    os << "--成绩单--" << endl;
    for (int i = 0; i < g.count; i++) {
        os << g.courses[i] << ": " << g.grades[i] << endl;
    }
    os << "average: " << (g.sum / g.count) << endl << endl;
    return os;
}

};

class Ability {
private:
    string skills[MAX_SKILLS];
    int count;

public:
    Ability() : count(0) {}

    void add(string skill) {
        if (count < MAX_SKILLS) {
            skills[count] = skill;
            count++;
        }
    }
}

friend ostream& operator<<(ostream& os, const Ability& a) {
    os << "--能力--" << endl;
    for (int i = 0; i < a.count; i++) {

```

```

    os << a.skills[i] << endl;

    }

    return os;

}

};

class Profile {

private:

    Student student;

    Grade grades;

    Ability abilities;

public:

    Profile() : student(), grades(), abilities() {}

    Profile(Student s, Grade g, Ability a) : student(s), grades(g), abilities(a) {}

    friend ostream& operator<<(ostream& os, const Profile& p) {

        os << "Student Growth Profile:" << endl;

        os << p.student;

        os << p.grades;

        os << p.abilities;

        return os;

    }

};

class InvalidIDException : public runtime_error {

public:

    InvalidIDException(const string& msg) : runtime_error(msg) {}

};

```

源文件（主函数）main.cpp

```

#include <iostream>

#include <vector>

#include "Student development file.h"

using namespace std;

float Grade::total_grade[MAX_COURSES] = { 0 };

```

```

int Grade::stu_count = 0;

void displayMenu() {
    cout << endl;
    cout << "==== 学生信息管理系统 =====> << endl;
    cout << "1. 添加学生" << endl;
    cout << "2. 显示所有学生信息" << endl;
    cout << "3. 查看具体科目的平均分" << endl;
    cout << "0. 退出" << endl;
    cout << "请选择操作: ";
}

void displayAllProfiles(const vector<Profile>& profiles) {
    if (profiles.empty()) {
        cout << "暂无学生信息。" << endl;
        return;
    }

    cout << "学生信息列表: " << endl;
    for (const auto& profile : profiles) {
        cout << profile;
    }
}

int main() {
    int stuCount = 0;
    int stu_n;

    cout << "请输入你要登记的学生总数: ";
    cin >> stu_n;

    vector<Profile> profiles;

    while (true) {
        displayMenu();
        int choice;

```

```

cin >> choice;

switch (choice) {
case 1: // 添加学生
if (stuCount < stu_n) {
cout << "请输入第" << (stuCount + 1) << "个学生的信息:" << endl;

    string id, name;
    int age, grade;

    // 捕获学号输入异常
    try {
        cout << "学号 姓名 年龄 年级: ";
        cin >> id >> name >> age >> grade;

        // 检查学号是否大于25000
        if (stoi(id) > 25000) {
throw InvalidIDException("学号不能大于 25000! ");
        }

        // 创建学生对象
Student student(id, name, age, grade);

        // 成绩输入
        float ma, ds, oop;
        cout << "请输入数学、数据结构、C++成绩: ";
        cin >> ma >> ds >> oop;
        Grade grades;

        grades.add("Mathematics", ma);
        grades.add("Data Structure", ds);
        grades.add("Object Oriented Programming", oop);

        // 能力输入
        Ability abilities;
        int n;
        cout << "请输入该学生的能力数与能力名称:" << endl;

        cin >> n;
        string ability;
        for (int j = 0; j < n; j++) {
            cin >> ability;
            abilities.add(ability);
        }

        profiles.push_back(Profile(student, grades, abilities));

        stuCount++;
        cout << "学生信息添加成功!" << endl;
    }

catch (const InvalidIDException& e) {

```

```

    cout << "输入错误: " << e.what() << endl;
    }
    catch (const exception& e) {
    cout << "发生异常: " << e.what() << endl;
    }
    }

else {
    cout << "学生数量已满，无法添加新学生!" << endl;
    }

break;

case 2: // 显示所有学生信息
    displayAllProfiles(profiles);
    break;

case 3: // 查看具体科目的平均分
    Grade::average_class();
    break;

case 0: // 退出
    cout << "退出系统。" << endl;
    return 0;

default:
    cout << "无效的选择，请重新输入。" << endl;
    }
}
}

```

附录二：实验结果截屏与实验成果分析

```
Microsoft Visual Studio 调试  x  +  v

请输入你要登记的学生总数：2

==== 学生信息管理系统 ====
1. 添加学生
2. 显示所有学生信息
3. 查看具体科目的平均分
0. 退出
请选择操作：1
请输入第1个学生的信息：
学号 姓名 年龄 年级：12345 jisoo 20 2
请输入数学、数据结构、C++成绩：95 90 100
请输入该学生的能力数与能力名称：
1 sing
学生信息添加成功！

==== 学生信息管理系统 ====
1. 添加学生
2. 显示所有学生信息
3. 查看具体科目的平均分
0. 退出
请选择操作：1
请输入第2个学生的信息：
学号 姓名 年龄 年级：12346 jennie 19 1
请输入数学、数据结构、C++成绩：98 95 93
请输入该学生的能力数与能力名称：
2 dance program
学生信息添加成功！

==== 学生信息管理系统 ====
1. 添加学生
2. 显示所有学生信息
3. 查看具体科目的平均分
0. 退出
请选择操作：2
学生信息列表：
Student Growth Profile:
--基本信息--
studentID: 12345
name: jisoo
age: 20
grade: 2

--成绩单--
Mathematics: 95
Data Structure: 90
Object Oriented Programming: 100
average: 95

--能力--
sing
Student Growth Profile:
--基本信息--
studentID: 12346
name: jennie
age: 19
grade: 1

--成绩单--
Mathematics: 98
Data Structure: 95
Object Oriented Programming: 93
average: 95.3333

--能力--
dance
program

==== 学生信息管理系统 ====
1. 添加学生
2. 显示所有学生信息
3. 查看具体科目的平均分
0. 退出
请选择操作：3

Class Grade:
--每门课程的平均分--
课程1的平均分：96.5
课程2的平均分：92.5
课程3的平均分：96.5

==== 学生信息管理系统 ====
1. 添加学生
2. 显示所有学生信息
3. 查看具体科目的平均分
0. 退出
请选择操作：0
退出系统。

D:\autumn\CS_Experience\OOP_Experience\实验
按任意键关闭此窗口...
```

结果分析：

1. 程序实现与功能分析

符合情况

学生信息管理系统： 程序成功实现了一个学生信息管理系统，可以进行学生信息的登记、查看所有学生信息以及查看具体科目的平均分。

学生类与功能： 使用了 `Person` 类和 `Student` 类，分别表示学生的个人信息（姓名和年龄）和学生的基本信息（学号、年级）。`Student` 类继承了 `Person` 类，这种继承关系符合面向对象设计的基本原则，提升了代码的复用性。

成绩管理： 通过 `Grade` 类管理学生的成绩，能够存储和计算每个学生的成绩以及计算全班的课程平均分，采用了静态成员变量 `total_grade` 来累计全班成绩。

能力管理： `Ability` 类用于管理学生的能力，可以存储多个能力（如编程能力、语言能力等）。

异常处理： 针对输入的学号（学号不能大于 25000）进行了有效的异常处理，通过 `InvalidIDException` 来捕获并处理异常，确保程序的稳定性。

灵活性： 利用 `vector<Profile>` 容器存储学生信息，提供了更灵活的存储方式，避免了固定大小数组的问题。

菜单与交互： 提供了简洁的菜单系统，用户可以选择不同的操作（添加学生、显示学生信息、查看平均分等），增强了系统的可操作性。

功能实现与设计符合

功能设计符合：

o **学生信息的添加与存储：** 可以动态添加学生信息（学号、姓名、成绩、能力等），符合设计目标。

o **学生信息展示：** 使用 `ostream` 重载运算符输出学生信息、成绩和能力信息，符合面向对象的设计理念，使得学生信息的显示更加简洁清晰。

o **平均分计算：** 通过 `Grade::average_class` 函数来计算并输出每门课程的平均分，符合需求。

异常处理设计符合：

o **异常处理机制：** 在学生学号输入时进行检查，如果学号超过 25000，会抛出 `InvalidIDException` 异常。通过 `try-catch` 块处理输入错误，提高了程序的健壮性。

2. 优点

面向对象设计： 程序实现了面向对象的设计思路，将学生信息、成绩、能力等封装到不同的类中，提高了代码的模块化和可维护性。

异常处理： 对学号输入进行了异常处理，避免了非法输入导致程序崩溃，提高了系统的健壮性。

灵活性： 使用 `vector<Profile>` 存储学生信息，避免了使用固定大小数组的问题，提升了存储的灵活性。

简单直观的用户界面： 提供了简洁的菜单，用户可以通过选择不同的操作项进行交互。

3. 不足与改进建议

学号校验过于简单： 当前学号的校验仅通过判断学号是否大于 25000 来进行，实际应用中，学号可能还需要更复杂的验证（如是否符合特定格式，是否已存在等）。可以加入更多的校验条件，提高系统的健壮性。

缺少学生删除和修改功能： 当前程序只能添加和查看学生信息，但缺少删除和修改学生信息的功能。可以增加这些功能，使得系统更加完善。